

Week 5 Progress Report

Student Name: Trent Carter

Course: Senior Capstone – Cybersecurity

Week: Week 5

Focus: Database Integration for Event Storage

Overview

During Week 5, the focus of the project shifted from data ingestion and normalization to integrating persistent storage for processed events. Building on the completed pipeline from Week 4, a SQLite database was implemented to store normalized log data in a structured and queryable format.

This step represents a major transition from temporary file-based storage (JSON) to a more scalable and realistic SIEM architecture. The database enables efficient querying, aggregation, and future detection logic, which are essential components of modern security monitoring systems.

Objectives

Integrate a database into the existing SIEM pipeline, design a schema for storing normalized events, implement database initialization and insertion logic, optimize insertion performance for large datasets, and validate stored data using SQL queries.

Implementation Details

Database Integration

A SQLite database (mini_siem.db) was created within the project directory to store all normalized events. SQLite was selected due to its simplicity, portability, and suitability for prototyping SIEM functionality.

A dedicated module (storage/db.py) was developed to handle all database interactions, including database initialization, table creation, and event insertion. This modular approach ensures separation of concerns and aligns with the architecture designed in previous weeks.

Schema Design

The events table was designed to align with the unified event schema developed earlier in the project. Each event includes structured fields such as event_id, timestamp, event_type, source, src_ip, user, message, and raw_log.

This schema allows multiple heterogeneous log sources, including system authentication logs and honeypot data, to be normalized into a consistent format for analysis.

Batch Insertion Optimization

Initial implementation used per-event database insertion, which resulted in significant performance issues when processing large datasets. To resolve this, batch insertion was implemented, allowing all events to be inserted in a single operation.

This optimization dramatically improved performance and reflects real-world log ingestion strategies used in SIEM systems.

Pipeline Integration

The database layer was integrated into the existing pipeline without disrupting prior functionality. The updated pipeline is as follows:

Raw Logs → Ingestion → Normalization → Database Storage

Normalized events are now both written to a JSONL file for validation and inserted into the SQLite database for structured storage and querying.

Results

The system successfully processed and stored over 1,081,531 normalized events in the database.

Program Output:

```
=== Mini SIEM Week 4: Ingestion + Normalization ===  
Normalized events written to: data/normalized/events.jsonl  
Total events: 1081531  
Parse failures: 0  
  
Top event types:  
  honeypot_event: 616176  
  amun_connection: 371317  
  ssh_disconnect: 46810  
  authlog_other: 20867  
  ssh_invalid_user: 12223  
  http_request: 7199  
  ssh_socket_error: 2403  
  ssh_connection_closed: 1974  
  ssh_no_identification: 969  
  cron_session_close: 797
```

Database Verification:

The following query was used to verify successful database insertion:

```
SELECT COUNT(*) FROM events;
```

Result:

Total events in DB: 1081531

Event Distribution Analysis:

The following query was used to analyze event distribution:

```
... SELECT event_type, COUNT(*)  
... FROM events  
... GROUP BY event_type  
... ORDER BY COUNT(*) DESC  
... LIMIT 10;
```

('honeypot_event', 616176)
('amun_connection', 371317)
('ssh_disconnect', 46810)
('authlog_other', 20867)
('ssh_invalid_user', 12223)
('http_request', 7199)
('ssh_socket_error', 2403)
('ssh_connection_closed', 1974)
('ssh_no_identification', 969)
('cron_session_close', 797)

These results confirm that the system is correctly processing and categorizing events from multiple data sources.

Challenges Encountered

Performance bottlenecks were encountered during initial database integration due to per-event insertion, which significantly slowed execution time. This was resolved through batch insertion.

Additionally, GUI-based database tools struggled to handle the large dataset size. Verification was instead performed using Python and SQL queries, which proved more reliable.

Handling over one million events required careful optimization to ensure efficient processing and storage.

Key Takeaways

Database integration is essential for scalable SIEM systems. Batch processing is critical when handling large volumes of log data. Structured storage enables efficient querying and analysis, and the system now closely resembles real-world SIEM data pipelines used in security operations.

Limitations of Current Dataset

The datasets used in this phase (auth.log and honeypot data) do not originate from the same environment and therefore do not directly correlate.

As a result, cross-source correlation and attribution are limited, and these datasets are not suitable for final detection and correlation analysis. However, they were intentionally used to simulate heterogeneous log ingestion and validate normalization and database functionality.

Future work will involve identifying datasets that share a common environment to support meaningful correlation and detection.

Next Steps (Week 6)

The next phase of the project will focus on detection logic and alert generation. Planned activities include developing rule-based detection mechanisms, identifying suspicious patterns such as brute force attempts and scanning behavior, generating alerts based on defined rules, and storing alerts within the database for further analysis.